Destructive Confirmation Input
Dots Menu
Drawer
Empty State
Entity
Error
Error Card
Feedback
Fieldset
File Tree
Filter Params
Gauge
Grid
Input
JSON View
Keyboard Input
Label
Load More Button
Loading Dots
Menu
MiddleTruncate
Modal
Multi Select
Note
Pagination
Phone
Pie Gauge

Entity

Displays up-to-two columns of content. The left column can contain arbitrary content, and the right column typically contains controls or actions related to the content in the left column.

Default



Evil Rabbit
Glenn Hitchcock (@gln)

Connected 1h ago

Hide code

```tsx
import { Avatar, Entity, EntityContent } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <Entity
      left={<Avatar size={32} username="evilrabbit" />}
      right={<p className="text-copy-14 text-gray-900">Connected 1h ago</p>}
    >
      <EntityContent
        description="Glenn Hitchcock (@gln)"
        fill
        title="Evil Rabbit"
      />
    </Entity>
  );
}
```

Entity with Skeleton

Hide code

```tsx
import { Entity, Skeleton } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <Entity>
      <div className="flex flex-col items-stretch justify-start gap-2 flex-1">
        <Skeleton height={20} width="100%" />
        <div className="flex flex-row items-center justify-start gap-2 flex-initial">
          <Skeleton height={20} width={70} />
          <Skeleton height={20} width={60} />
          <Skeleton height={20} width={68} />
        </div>
      </div>
    </Entity>
  );
}
```

Entity with List

GitHub Desktop on MacBook Pro
Last used just now — Decline

VS Code on Windows 11
Last used 10min ago — Decline

Terminal on Ubuntu 24.04
Last used 25min ago — Decline

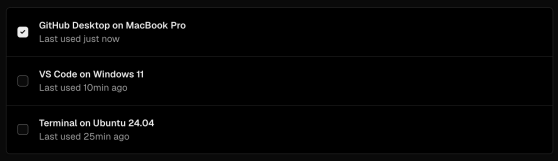
Hide code

```tsx
import {
  Button,
  Entity,
  EntityList,
  EntityContent,
} from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <EntityList>
      <Entity
        as="li"
        right={
          <Button size="small" variant="secondary">
            Decline
          </Button>
        }
      >
        <EntityContent
          description="Last used just now"
          title="GitHub Desktop on MacBook Pro"
        />
      </Entity>
      <Entity
        as="li"
        right={
          <Button size="small" variant="secondary">
            Decline
          </Button>
        }
      >
        <EntityContent
          description="Last used 10min ago"
          title="VS Code on Windows 11"
        />
      </Entity>
      <Entity
        as="li"
        right={
          <Button size="small" variant="secondary">
            Decline
          </Button>
```

```
        }
      >
        <EntityContent
          description="Last used 25min ago"
          title="Terminal on Ubuntu 24.04"
        />
      </Entity>
    </EntityList>
  );
}
```

Entity with List and Checkbox



Hide code

```
import { useState } from 'react';
import {
  Checkbox,
  Entity,
  EntityList,
  EntityContent,
} from '@vercel/geistcn/components';
import type { JSX } from 'react';

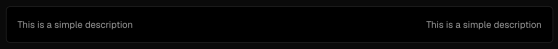
export function Component(): JSX.Element {
  const items = [
    {
      id: 'github',
      title: 'GitHub Desktop on MacBook Pro',
      description: 'Last used just now',
    },
    {
      id: 'vscode',
      title: 'VS Code on Windows 11',
      description: 'Last used 10min ago',
    },
    {
      id: 'terminal',
      title: 'Terminal on Ubuntu 24.04',
      description: 'Last used 25min ago',
    },
  ];

  const [checkedStates, setCheckedStates] = useState<Record<string, boolean>>({
    github: true,
    vscode: false,
    terminal: false,
  });

  const handleCheckboxChange = (id: string): void => {
    setCheckedStates((prev) => ({
      ...prev,
      [id]: !prev[id],
    }));
  };

  return (
    <EntityList>
      {items.map((item) => (
        <Entity
          key={item.id}
          as="button"
          onClick={() => handleCheckboxChange(item.id)}
          left={
            <Checkbox
              aria-label={item.title}
              checked={checkedStates[item.id as string]}
              onChange={() => handleCheckboxChange(item.id)}
            />
          }
        >
          <EntityContent description={item.description} title={item.title} />
        </Entity>
      ))}
    </EntityList>
  );
}
```

Entity with Fill



Hide code

```
import { Entity, EntityContent, EntityList } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <EntityList>
      <Entity>
        <EntityContent fill description="This is a simple description" />
        <EntityContent description="This is a simple description" />
      </Entity>
    </EntityList>
  );
}
```

Entity with Column ClassNames



Hide code

```
import {
  Avatar,
  Entity,
```

```
  EntityContent,
  EntityList,
} from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <EntityList>
      <Entity
        left={<Avatar placeholder size={50} />}
        leftClassName="border border-dashed border-gray-300 rounded-md p-2"
        right={
          <span className="text-copy-14 text-gray-900">[some action]</span>
        }
        rightClassName="border border-dashed border-gray-300 rounded-md p-2"
      >
        <EntityContent description="Entity with dashed borders" />
      </Entity>
    </EntityList>
  );
}
```

Best Practices

When to use

- Use `<Entity>` for a row of descriptive content paired with one or two controls (member rows, integration rows, domain rows).
- For tabular data with sortable columns and shared row shape, use Table instead.
- For a static key/value metadata block on a detail page, use Description.

Behavior

- The right column holds at most one or two controls. If the row needs more, move secondary actions into a Dots Menu.
- For multi-select rows, the leading Checkbox carries `aria-label="Select {entity name}"` so the row is selectable without relying on the visual label.
- Render the Skeleton variant (`entity-with-skeleton`) during load instead of an empty row, and swap to real content once data resolves.

Content

- Lead the left column with a scannable identifier: an Avatar or icon, a Title Case label, then sentence-case secondary metadata (`Member since Mar 14, 2026`).
- Keep right-column buttons Verb + Noun (`Remove Member`, `Resend Invite`). Bare verbs like `Remove` or `Confirm` lose context once the row scrolls offscreen.